



GHIX UI Automation Framework

Complete Test Execution Flow Guide

Cucumber.js

Playwright

Allure Reports



High-Level Execution Architecture



1. Entry Point: package.json

The test execution begins when you run `npm test`. The script defined in `package.json` orchestrates the entire process:

```
"test": ".\\node_modules\\.bin\\env-cmd -f .\\environment.env cucumber-  
js test --format ./tests/common.utils/reporter.js"
```

What this command does:

- 1 **env-cmd -f environment.env** - Loads all environment variables from `environment.env` into `process.env` BEFORE cucumber runs
- 2 **cucumber-js test** - Starts the Cucumber test runner, which automatically reads `cucumber.js` for configuration
- 3 **--format ./tests/common.utils/reporter.js** - Uses the Allure reporter for generating test results

2. Environment Configuration: environment.env

This file contains all configurable environment variables that control test execution:

```
BROWSER=chromium state=GA applicationYear=2026
tagExpression=@GA_OEP_Regression SERVER=ga2auto.eng.vimo.com
TIMEOUT_MAX=50000 TIMEOUT_MIN=30000
eksScdfLaunchUrl=scdf.ghixqa.com/scdf/tasks/executions/launch
enableReportportal=false UPDATE_SNAPSHOTS=false accessibilitytest=false
visualregressiontest=false
```

Variable	Purpose
BROWSER	Which browser to use (chromium/firefox)
state	State profile for testing (GA, NV, PA, VA, etc.)
applicationYear	The OEP year for plan testing
tagExpression	Which Cucumber test tags to run
SERVER	Target server hostname
TIMEOUT_MAX/MIN	Timeout configurations for waits
enableReportportal	Toggle ReportPortal integration
accessibilitytest	Toggle accessibility testing
visualregressiontest	Toggle visual regression testing

3. Cucumber Configuration: cucumber.js

This file tells Cucumber where to find features, step definitions, and hooks:

```
const common = ` --require ./tests/common.utils/Hooks.js --require
./tests/stepDefs/**/*.js --format ./tests/common.utils/reporter.js --
tags '@GA_OEP_Regression' --parallel 0 --retry 0 `; const features = [
'./tests/features/!(UserAccountManagement)/**/*.feature', ];
module.exports = { default: `${common} ${features.map(p =>
`"${p}"`).join(' ')} `};
```

Option	What it does
<code>--require</code> <code>./tests/common.utils/Hooks.js</code>	Loads hooks FIRST - Sets up entire lifecycle
<code>--require ./tests/stepDefs/**/*.js</code>	Loads all step definitions from subdirectories
<code>--format</code> <code>./tests/common.utils/reporter.js</code>	Uses Allure for reporting
<code>--tags '@GA_OEP_Regression'</code>	Only runs scenarios tagged with this
<code>--parallel 0</code>	Runs tests sequentially (no parallelization)
<code>--retry 0</code>	No retries on failure

4. Hooks Lifecycle: Hooks.js

✦ **This is the heart of the framework!** Hooks.js controls the entire test execution lifecycle including browser management, state initialization, and cleanup.

4.1 BeforeAll Hook (Runs Once Per Suite)

Initializes the entire test suite before any scenario runs:

```
BeforeAll(async function () { // Load environment variables into global
scope global.browserOption = process.env.BROWSER; global.state =
process.env.state; global.server = process.env.SERVER;
global.tagExpression = process.env.tagExpression;
global.applicationYear = process.env.applicationYear; // Launch browser
based on execution mode if (process.env.JENKINS === 'true') { //
Headless mode for CI/CD global.pwBrowser = await
chromium.launch({headless: true}); } else { // Headed mode for local
development global.pwBrowser = await chromium.launch({headless:
false}); } // Server health checks (for Dev_Smoke tag) // Server
configuration validation })
```

- Reads ALL environment variables into global scope
- Launches Playwright browser (headed for local, headless for Jenkins)
- Performs server health checks if Dev_Smoke tag detected
- Validates server configuration against database

4.2 Before Hook (Runs Before EACH Scenario)

Sets up a fresh environment for each test scenario:

```
Before(async function (scenario) { // Reset all scenario-specific
  global variables global.softAssertions = []; global.cucumberSteps = [];
  // Create new browser context and page (isolation) global.context =
  await global.pwBrowser.newContext({ viewport: null, ignoreHTTPSErrors:
  true }); global.page = await global.context.newPage(); global.url =
  'https://' + global.server + '/hix/'; // Create Browser wrapper
  global.browser = new (require('../base/Browser'))(global.page); // JIRA
  integration - extract test IDs let storyJIRAs = await
  propertyReader.getStoryJIRAs(scenario.pickle.name); })
```

- Resets all scenario-specific global state
- Creates fresh browser context and page (isolation between scenarios)
- Sets up the URL from environment
- Wraps Playwright page in custom Browser class
- Extracts JIRA IDs and links them to reports

4.3 BeforeStep / AfterStep Hooks

Handles logging and error capturing at the step level:

```
BeforeStep(async function (scenario) { console.log("Start Step -- " +
  scenario.pickleStep.text);
  global.cucumberSteps.push(scenario.pickleStep.text); global.testLogs =
  []; }); AfterStep(async function (scenario) { if (isFailedStep ||
  hasSoftAssertions || hasLogs) { // Attach logs // Take screenshots on
  failure // Handle soft assertions } });
```

4.4 After Hook (Runs After EACH Scenario)

Cleans up resources and reports results:

```
After(async function (scenario) { // Close page and context if
(global.page) await global.page.close(); if (global.context) await
global.context.close(); // ReportPortal integration - insert test
results to DB if (process.env.enableReportportal === 'true') { await
commonfunction.insertDataToTableTestAutomationExecution(scenario); } //
Cleanup globals to prevent memory leaks global.page = null;
global.context = null; global.updateDataJson = null; });
```

4.5 AfterAll Hook (Runs Once at End)

Closes browser and generates final reports:

```
AfterAll(async function () { // Close browser if (global.pwBrowser)
await global.pwBrowser.close(); // Generate Visual Regression reports
(if enabled) if (enableVisualRegressionTest) {
visualUtils.generateVisualReportForState(state); } // Generate
Accessibility reports (if enabled) if (enableAccessibilityTest) {
accessibilityUtils.generateAccessibilityReportForState(state); } });
```



5. Reporter: reporter.js

Creates Allure reports in the `./allure-results` directory:

```
const {AllureRuntime} = require('allure-js-commons'); const
{CucumberJSAllureFormatter} = require('allure-cucumberjs');
module.exports = class extends CucumberJSAllureFormatter {
  constructor(options) { const runtime = new AllureRuntime({ resultsDir:
    "./allure-results" }); super(options, runtime,
    options.parsedArgvOptions); } };
```

View reports with: `npm run report:view`

 6. Key Global Variables

Variable	Set In	Purpose
<code>global.pwBrowser</code>	BeforeAll	Raw Playwright browser instance
<code>global.context</code>	Before	Browser context (cookies, storage)
<code>global.page</code>	Before	Playwright page object
<code>global.browser</code>	Before	Custom Browser wrapper class
<code>global.url</code>	Before	Base URL for tests
<code>global.state</code>	BeforeAll	Current state profile (GA, NV, etc.)
<code>global.server</code>	BeforeAll	Server hostname
<code>global.updateDataJson</code>	Before	Test data storage

 Complete Flow Summary

- 1. `npm test`
 - ↳ `env-cmd` loads `environment.env` → Variables in `process.env`
- 2. `cucumber-js` starts
 - ↳ Reads `cucumber.js` config
 - ↳ `--require Hooks.js` (loaded first)
 - ↳ `--require stepDefs/**/*.js`
 - ↳ `--tags` determines which features run
- 3. `BeforeAll()`
 - ↳ Launch Playwright browser (chromium/firefox)
 - ↳ Initialize global state from `process.env`

- ↳ Health checks (optional)

4. For each Feature/Scenario matching tags:

- ↳ Before() - Create new page/context, reset state

- ↳ For each Step:

- ↳ BeforeStep() - Log step start

- ↳ Execute step definition

- ↳ AfterStep() - Screenshot on failure, attach logs

- ↳ After() - Close page, report results, cleanup

5. AfterAll()

- ↳ Close browser

- ↳ Generate visual/accessibility reports

- ↳ Write allure-results

GHIX UI Automation Framework

Documentation generated for test execution understanding

To save as PDF: Open this file in browser → Press Ctrl+P → Save as PDF